balancing_bach_smt2026

Code for the SMT project Balancing Bach

What this project does

Bach's *Well-Tempered Clavier* (WTC) has a prelude and fugue in each of the 24 major and minor keys, in two books: 48 pairs in total (BWV 846–893). This code counts how often each harmonic interval (two notes sounding together) occurs in the WTC, separately for every pair of pitch classes. For example, the major third C–E is counted separately from the major third D–F♯. Melodic intervals (between successive notes) are not counted.

Counts like these matter for **temperament**. On a keyboard every key has one fixed tuning, so a tuning system decides how pure or impure each specific third or fifth sounds. Knowing which intervals Bach uses most tells us which ones a tuning should favour.

Files

File	Contents
`scores_xml/`	The 48 preludes and fugues as MusicXML files, named by BWV number (`BWV_0846.xml` = Book I, C major … `BWV_0893.xml` = Book II, B minor). Each file holds one prelude–fugue pair.
`temperament_functions.py`	The functions that parse the scores, count the intervals and print the results.
`intervals-tally.ipynb`	Notebook that runs the analysis on all 48 files and prints the result.
`wtc_frequencies.json`	The output of the notebook (see below).
`docs/how_the_tally_works.md`	A step-by-step example of the counting for one measure, with figures.

How the counting works

For a worked example with music notation, see How the tally works.

How an interval is represented

Each interval is stored as a tuple

```
(lower_pitch_class, upper_pitch_class, interval_class)
```

- **Pitch classes** are numbers 0–11: C = 0, C♯/D♭ = 1, D = 2, … B = 11. The octave and the spelling are thrown away: D♯ and E♭ are both 3.

- **Interval class** is the number of semitones from the lower note up to the higher note, reduced modulo 12. So a major tenth counts as a major third (4), and a unison (two voices on the same note) and an octave both count as 0.

Example: E3 below C5 gives `(4, 0, 8)`, a minor sixth from E up to C.

Finding the intervals

The score is *chordified*: all voices are combined into a series of vertical slices, and a new slice begins every time any note starts or ends. Within each slice, every pair of notes counts as one interval.

Weighting

Each occurrence adds a weight to the tally rather than simply 1:

- `durationWeight=True`: weight by the length of the slice in quarter notes.
- `metricWeight=True`: multiply by music21's *beat strength*: 1.0 on the downbeat, smaller values on weaker beats and off-beats.

With both set to `False`, every occurrence counts as exactly 1.

Normalisation

All pieces add to one shared tally. With `total_weight=1.0` the result is scaled so that all values sum to 1, so each value is the *share* of that interval in the whole corpus.

Main functions

- `get_semitone_interval(note1, note2)`: returns `(interval_class, lower_pc, upper_pc)`.
- `analyze_intervals(score, pair_frequencies, ...)`: adds the intervals of one score to `pair_frequencies`.
- `process_scores(score_files, ...)`: parses all files, tallies them and optionally normalises the result.
- `pprint_interval_data(pair_frequencies, number_of_flats=3)`: prints the tally sorted by interval, by lower note and by weight. Because only pitch classes are stored, a spelling has to be chosen for printing. `number_of_flats=3` gives C C♯ D E♭ E F F♯ G A♭ A B♭ B. This affects only the display, not how the note was written in the score.

Output file

`wtc_frequencies.json` maps each interval tuple, written as a string, to its weight:

```
{
  "(0, 4, 4)": 0.008568,
  ...
}
```

To read it back in Python:

```
import json, ast
with open('wtc_frequencies.json') as f:
    freqs = {ast.literal_eval(k): v for k, v in json.load(f).items()}
```

Running the code

Requirements: Python 3.11 or later and music21 (tested with version 8.3).

```
pip install music21 jupyter
jupyter notebook intervals-tally.ipynb
```

Parsing and chordifying all 48 files takes a few minutes.